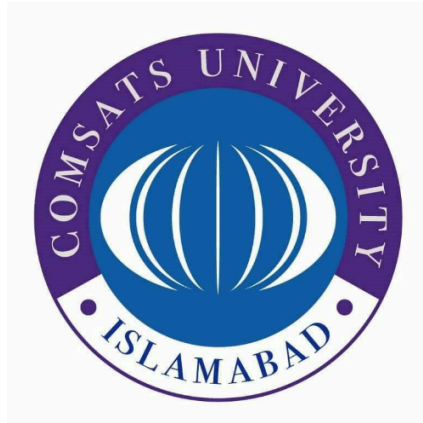




# **PARALLEL & DISTRIBUTED COMPUTING**

## **LAB ASSIGNMENT #1**

**PREPARED BY**

**M Mukaram Awan**  
SP22-BCS-141

**SUBMITTED TO**

**Sir Akhzar Nazeer**

**Q1: Why does choosing a block size that is not a multiple of 32 (warp size) lead to underutilization of GPU hardware resources?**

GPUs execute threads in groups of 32 called warps. If the block size is not a multiple of 32, the last warp in the block may not be fully populated with active threads, causing some threads in that warp to remain idle.

Example:

For a block size of 40 threads, the GPU allocates 2 warps (64 threads). Out of these, only 40 threads are active and 24 threads are idle, wasting GPU execution lanes and leading to hardware underutilization.

**Q2: Explain how occupancy of an SM (Streaming Multiprocessor) depends on block size and threads per block.**

Occupancy is the ratio of active warps per SM to the maximum number of warps supported by that SM.

- If **block size is too small**, fewer warps are active, resulting in **low occupancy**.
- If **block size is too large**, it may use excessive registers or shared memory, limiting the number of concurrent blocks on an SM and **reducing occupancy**.  
Therefore, optimal performance is achieved by **balancing thread count, register usage, and memory per block** to maintain high occupancy.

**Q3: CUDA Image Inversion**

```
import numpy as np

from numba import cuda

import cv2

import time

# CUDA kernel for image inversion

@cuda.jit

def invert_image(input_img, output_img):
```

```

x, y = cuda.grid(2)

if x < input_img.shape[0] and y < input_img.shape[1]:

    output_img[x, y] = 255 - input_img[x, y]

def run_inversion(input_img, block_size):

    output_img = np.zeros_like(input_img)

    threadsperblock = block_size

    blockspergrid_x = (input_img.shape[0] + threadsperblock[0] - 1) // threadsperblock[0]

    blockspergrid_y = (input_img.shape[1] + threadsperblock[1] - 1) // threadsperblock[1]

    blockspergrid = (blockspergrid_x, blockspergrid_y)

    d_input = cuda.to_device(input_img)

    d_output = cuda.to_device(output_img)

    start = time.time()

    invert_image[blockspergrid, threadsperblock](d_input, d_output)

    cuda.synchronize()

    end = time.time()

    result = d_output.copy_to_host()

    return result, end - start

if __name__ == "__main__":

    img = cv2.imread("mukaram.jpg", cv2.IMREAD_GRAYSCALE)

```

```
block_sizes = [(8, 8), (16, 16), (32, 32)]
```

```
for bs in block_sizes:
```

```
    output, exec_time = run_inversion(img, bs)
```

```
    print(f"Block Size {bs}: Execution Time = {exec_time:.6f} sec")
```

```
    cv2.imwrite(f"inverted_{bs[0]}x{bs[1]}.png", output)
```

## Q:3 Analysis

### Execution Configurations:

- **Case A:** 64 threads per block → **(8×8)**
- **Case B:** 256 threads per block → **(16×16)**
- **Case C:** 1024 threads per block → **(32×32)**

### Observation:

- **Case B** (256 threads) usually performs **best**.

### Reasoning:

- **Case A (64 threads):**  
Underutilizes the GPU. Fewer warps per block → Lower throughput.
- **Case C (1024 threads):**  
Consumes excessive registers and shared memory, reducing the number of active blocks per SM → **Lower occupancy**.
- **Case B (256 threads):**  
Provides a **balanced configuration**—enough threads for good utilization without exceeding hardware limits, ensuring **high occupancy and efficient performance**.

**generic Code which automatically utilizes maximum or more suitable block sizes and thread sizes according to the requirement**

```
def get_optimal_block_size(img_shape):  
    h, w = img_shape  
    area = h * w  
    if area > 1_000_000:  
        return (16, 16)  
    elif area > 10_000:  
        return (8, 8)  
    else:  
        return (4, 4)
```

**Q: Why does increasing the number of threads per block not always improve performance?**

**Answer:**

- **Larger blocks** use more **registers** and **shared memory**.
- This restricts how many blocks can run **concurrently** on an SM.
- The **GPU scheduler** may not be able to launch multiple large blocks at once, lowering **concurrency** and **overall performance**.

Hence, **bigger blocks ≠ better performance**. A **balanced block size** ensures high utilization and occupancy without exhausting hardware resources.